ZUNFT

The Complete Beginner's Guide

What we have built so far, how the app works, and what comes next - explained for someone who has never worked with any of this technology.

Status: Steps 1 to 6 of the build plan are finished and tested. Step 7 (a real test with real keys and phones, then release) is next.
Written 3 October 2026.

How to read this guide

This guide assumes **zero technical knowledge**. Every technical word is explained the first time it appears, and there is a glossary (Chapter 3) you can flip back to at any time.

You will see four kinds of coloured boxes:

> **IN PLAIN WORDS**
> The simplest possible explanation of the idea.

> **ANALOGY**
> A comparison with everyday life, so the idea sticks.

> **GOOD NEWS**
> Something that works well or was verified.

> **BE CAREFUL**
> A limit, a risk, or something not yet verified. We are honest about these.

Contents

1. **The idea** - what Zunft is and who it is for
2. **The big picture** - the three pieces and how they talk
3. **Glossary** - every technical word, in plain English
4. **Where we started** - what the project looked like when I first read it
5. **Decisions we made together**
6. **What was built, step by step** (Steps 1 to 6)
7. **How the app works** - a walk through every screen and data flow
8. **The money and tax rules inside the app**
9. **Security** - how customer data is protected
10. **The files** - a map of the project folder
11. **Testing** - how we know it works (and what we could not check)
12. **What is still open** - honest limits
13. **What comes next** - Step 7, and what you need to do
14. **Your questions, answered** - team, privacy, offline, one invoice per note
15. **Your next steps** - a simple checklist for the first real test

1. The idea

Zunft (German for "guild", the traditional association of craftsmen) is a phone app for small German craft businesses - *Handwerksbetriebe*. The first customer we are designing for is an electrician with 1 to 10 employees in Hamburg.

The problem it solves

Picture Thorsten, an electrician. He spends the day on building sites: installing sockets, fixing fuse boxes. When he gets home at 7 pm he still has to write quotes and invoices - typing numbers into a computer after a long day. Many craftsmen hate this and lose hours (and money) every week. Some forget to invoice at all.

The solution

1. Thorsten speaks into his phone on the job site
"Customer Müller, new socket in the basement, two hours, material about 40 euros."

▼

2. The app turns speech into text, then an AI turns text into a structured draft
Customer name, work items, hours, material, prices - all separated correctly.

▼

3. Thorsten checks the draft on screen, fixes anything, taps "PDF"
A professional German invoice, ready to send by WhatsApp or e-mail.

Total effort: about one minute instead of twenty.

Why German craft businesses specifically?

- **German invoice rules are strict.** An invoice needs specific information (address, tax number, invoice number, and so on). The app knows these rules so the craftsman does not have to.
- **A tax rule called § 35a EStG.** Private customers can deduct part of a craftsman's bill from their income tax - but *only the labour part, not the material*. So the invoice must show labour and material separately. Done wrong, the customer loses money and complains. The app separates them automatically.
- **Craftsmen work where there is bad network** (basements, new buildings). So the app must work *without internet* and catch up later.

The long-term vision (three layers)

Layer	What	Status
1. The hook	Voice note to quote/invoice PDF (what we are building now, "Phase 1").	In progress
2. Overflow network	When a business is fully booked it can pass jobs to nearby colleagues, with a referral fee ("Phase 3").	Later
3. Compliance	Legally tamper-proof records (GoBD) and electronic invoice formats (ZUGFeRD / XRechnung) ("Phase 4").	Later

IN PLAIN WORDS

We are only finishing Layer 1. Layers 2 and 3 are deliberately postponed. The database already has a few small "hooks" for them so we do not have to rebuild later.

2. The big picture

The whole system has **three pieces**. Understanding these three is 80% of understanding the project.

PIECE 1 - The phone app (Flutter)
What Thorsten holds in his hand. Screens, buttons, the microphone, a small private database on the phone. Lives in the folder `zunft_app/`.

▲▼ talks over the internet ▲▼

PIECE 2 - The cloud (Supabase)
A rented online "back room": the main database, the login system, and file storage for recordings and PDFs. Lives in the folder `supabase/`.

▲▼ asks for help ▲▼

PIECE 3a - Deepgram
Specialist company that turns **speech into written text** (German).

PIECE 3b - Google Gemini
AI that **reads that text and understands it**: who is the customer, what work, what price.

ANALOGY: A RESTAURANT

The **phone app is the dining room** - what the guest sees and touches. **Supabase is the kitchen and pantry** - it stores everything and does the work guests should not see. The **"Edge Function"** (explained later) is the **head chef** who takes an order and calls two specialists: Deepgram is the *listener* who writes down exactly what was said, Gemini is the *clerk* who understands the note and fills out the form correctly.

Why not put everything on the phone?

- **Secret keys.** Deepgram and Gemini charge money per use. Their passwords (called *API keys*) must never be stored in the app, because anyone can pull them out of an app. They live only in the cloud piece.
- **Shared data.** If the business has several employees, they all need to see the same customers and jobs. That needs one central copy (the cloud).
- **Backup.** Phones get lost and broken. The cloud copy survives.

Why not put everything in the cloud?

Because job sites have bad reception. So the phone keeps its **own small copy** of the data and works with it. When the internet is there, the two copies are synchronised ("sync"). This is called an **offline-first** app, and it shapes a lot of the work in Step 4.

The journey of one voice note

1. **Thorsten records.** The phone saves an audio file and uploads it to Supabase storage in a folder named after his business.
2. **The app calls the Edge Function** ("please process this recording") and shows its login proof.
3. **The Edge Function checks who is asking**, finds his business, and downloads the audio.
4. **Deepgram** turns the audio into German text.
5. **Gemini** reads the text, the list of existing customers and the business's usual hourly rate, and returns a structured draft.
6. **The Edge Function double-checks the AI's answer** (AI can make mistakes), calculates the tax split and totals, saves a job and an audit record, and sends the draft back.
7. **The phone shows the draft.** Thorsten corrects anything, then creates the PDF.

> **WHERE WE ARE**
>
> Every stage of this journey now exists in code, including the last one: a checked draft becomes a numbered, locked invoice with a proper PDF. What has **not** happened yet is one real end-to-end run with real keys on a real phone - see Chapter 11.

3. Glossary

Flip back here whenever a word is unfamiliar.

Word	Plain-English meaning
App	A program on your phone.
Flutter	A toolkit from Google for building one app that runs on **both Android and iPhone** (and Windows) from the same code. Saves building everything twice.
Dart	The programming language Flutter apps are written in.
Code / source code	The written instructions that make a program do things.
Database	An organised electronic filing cabinet.
Table	One drawer of that cabinet, like a spreadsheet: *customers*, *jobs*, *documents*. Each row is one customer; each column is one detail (name, phone...).
Postgres / SQL	Postgres is the database software Supabase uses. SQL is the language you use to talk to it.
Supabase	A company that rents out a ready-made "back room": database + login system + file storage. Saves us from building our own servers.
Server / cloud	A computer somewhere else that is always on and reachable over the internet.
Backend / frontend	Frontend = what you see (the app). Backend = the hidden part behind it (the cloud).
API	A "menu" a service offers to other programs: "send me audio, I send back text".
API key	A long secret password that identifies you to a service and lets them bill you. **Must never be public.**
Edge Function	A small program that lives in the cloud and runs only when asked. Ours is called `process-capture`. It is the "head chef".
AI model / LLM	A program (here: Google Gemini) that can read text and produce text or structured answers. "Low-cost Gemini" means one of their cheaper, faster versions.
Speech-to-text (STT)	Turning a recording into written words. Deepgram does this.
Authentication (login)	Proving who you are (e-mail + password).
Login token (JWT)	A digital "wristband" you receive after logging in. The app shows it with every request so the server knows who is asking.

RLS (row-level security)	Locks *inside* the database: even if two businesses use the same drawer, each can only see its own rows. Like a hotel key card that opens only your room.
Migration	A saved list of changes to the database layout ("add this table, add this lock"). Keeps every copy of the database identical.
Drift / SQLite	SQLite is a tiny database that lives inside the phone. Drift is the helper that makes it pleasant to use from Flutter.
Offline-first	The app works from the phone's own copy of the data and syncs later, so a missing network never blocks work.
Sync	Making the phone's copy and the cloud copy agree: send local changes up, bring remote changes down.
Outbox	A "to-send" tray on the phone. Every change is dropped in it and removed only once the cloud confirms receipt.
Transaction	A group of database steps that either **all** happen or **none** do - like paying at a till: the money and the receipt together, never one without the other. Invoice numbers rely on this.
Embedded font	A typeface stored *inside* the PDF, so letters like ä ö ü ß and the euro sign look the same on every device.
Queue	A waiting line. Voice notes wait in a queue on the phone until they can be processed, one after the other, oldest first.
Idempotent	"Safe to repeat". If the same request arrives twice, the result is the same as if it had arrived once. Every voice note carries a unique number so the cloud never processes it twice.
Riverpod	A Flutter helper that hands the right data to the right screen and refreshes screens when data changes.
Widget	One building block of a screen: a button, a text, a list.
Test	A small program that checks another program behaves correctly. We wrote 205. Like a checklist an inspector ticks automatically.
Git	A "save point" system for code - lets you go back and see who changed what. Set up but nothing saved into it yet.
§ 35a EStG	The German tax law paragraph that lets private customers deduct craftsman *labour* costs (not material) from their taxes.
GoBD	German rules for keeping business records tamper-proof. Finished invoices must not be silently changed.
DIN 5008	The German standard for how a business letter or invoice is laid out.

Cents	We store every price as a whole number of cents (65,00 EUR is stored as 6500). Computers make tiny errors with decimals like 0.1 + 0.2, so money never uses decimals internally.

4. Where we started

When I first read your folder, it already held a detailed plan (`ZUNFT_BUILD_PLAN.md`) and a good amount of code. Think of it as a house with the blueprint and the walls up, but no electricity or plumbing connected.

What already existed and was good

- A **well thought-out plan** with a product description, sprint schedule and a "definition of done".
- A **database design** (six tables: businesses, staff profiles, customers, jobs, capture logs, documents) using whole cents for money.
- A **draft cloud function** for the voice-note processing.
- A **good-looking demo** of the app: a record button, a review screen where you edit line items, and a PDF preview in German invoice layout.
- Correct money calculations with **a few tests**.

What was missing or broken

Problem	Why it mattered
The app was not connected to anything	Everything on screen was made-up demo data. No login, no saving, no cloud.
The record button was fake	It toggled a colour and counted nothing. No audio was ever recorded.
The cloud function had no security	Anyone who found its address could read other businesses' data and run up your AI bill.
Weak database locks	Staff could change each other's roles; invoice numbers could clash or repeat.
No phone-specific folders	The project could only be tested on a PC, not built for Android or iPhone.
Wrong tax handling of flat fees	A call-out fee was treated as material, which would cost customers tax savings.
Invoice details hard-wired	Same invoice number, company and address on every PDF. **Fixed in Step 6.**
Letters like ä ö ü ß in the PDF	The PDF used a font that cannot draw them. **Fixed in Step 6.**
No history, README or setup notes	A stranger (or you in six months) could not run or understand it.

5. Decisions we made together

Question	Decision	Reason
Which AI?	**Google Gemini** (low-cost "Flash-Lite" tier to begin with) instead of Claude.	You asked for low cost. The model name is a setting (`LLM_MODEL`), so it can be switched later without rewriting anything.
Which phones?	**Android and iPhone**	Flutter handles both. Note: building the iPhone version requires a Mac computer (Apple's rule).
Is a call-out fee (Anfahrtspauschale) "labour" for § 35a?	**Yes - counts as labour.** Only material is excluded.	You have no tax adviser, so I decided from my knowledge of the official guidance that travel costs are part of the labour cost. **PLEASE VERIFY** This is from memory and not tax advice. Check the current BMF letter on § 35a before sending real invoices.
Speech-to-text vendor	Keep **Deepgram** for now.	The plan benchmarks it for German trade words. Gemini can also listen to audio directly, which would remove a vendor - we will compare them with real recordings.

6. What was built, step by step

The plan had seven steps. Six are complete. Each section below says **what** was done, **why**, and **how it works**.

Step 1 - Foundation done

Goal: make the project a proper, buildable, documented project.

- **Git created.** The folder is now a Git project, with a `.gitignore` list that keeps junk (build output) and **secret keys** out of the history. *Nothing has been "committed" (saved) yet - I only commit when you ask.*
- **Phone folders added** (`android/` , `ios/` , `windows/`) by running Flutter's own scaffolding command.
- **A real bug fixed:** the colour theme used a name that the current Flutter no longer accepts, so the app did not compile. Fixed.
- **README and `.env.example`** written: how to start the project and which secret settings are needed (with the safe rule: secrets only ever in the cloud, never in the app).
- **Internet permission added for Android.** Android's default template only allows internet access in *test* builds. A real release build would have been unable to reach the cloud at all.

Step 2 - Backend hardening done

Goal: make the cloud side safe and correct. Two parts: the Edge Function, and the database.

2A. THE EDGE FUNCTION `PROCESS-CAPTURE`

> **ANALOGY: THE HEAD CHEF**
> Before, the chef took orders from anyone who shouted through the kitchen door, and trusted whatever the customer said about who they were. Now there is a **doorman**: he checks the wristband (login token), looks up which business you belong to *himself*, and only lets you into your own pantry.

Improvement	What it means
Login required	No valid login token: refused (error 401). The business is found from the user's profile, **not** from anything the caller claims.
Acts as the user	It no longer uses the all-powerful "master key". Every database action runs with the user's own limited rights, so the database locks (RLS) protect each business.
Folder rule for recordings	A recording must live under `<business id>/...`. Nobody can ask it to transcribe another business's audio.
Size limits	Audio max 10 MB, text max 4,000 characters - protects against runaway bills.

Switched to Gemini	Asks Gemini for an answer in a strict JSON shape. The model name is a setting.
Never trust the AI blindly	A checker (`_shared/draft.ts`) rejects nonsense: unknown item types, negative quantities, absurd prices, or a customer ID the AI invented. If the answer is bad it retries once, then returns a friendly error.
Prompt-injection defence	The spoken text could contain "ignore your rules and..." (someone could say it on purpose). The text is wrapped in labelled tags and the AI is told it is only *data*, never instructions.
No leaking	Errors shown to the app are short and generic; technical details go to the server log only.
Bugs fixed	Empty speech result no longer crashes it; audio type is detected from the file; failing to save a job now stops with an error instead of silently continuing.

2B. THE DATABASE MIGRATION

A **migration** is a saved set of database changes. We kept the original and added a second one, `20261002000000_harden_schema.sql`.

Improvement	What it means
Safe invoice numbers	Numbers look like `RE-2026-0001` , `RE-2026-0002` ... The old method ("count existing + 1") could give two invoices the same number if two people saved at once. The new method keeps a **counter per business, type and year** that is locked while it is being increased - like a single ticket machine with one person at a time. If a save fails, the number is handed back, so there are **no gaps** (German tax law dislikes gaps).
Better locks (RLS)	Old: one broad rule per table. New: separate rules for reading, adding, changing and deleting. A job cannot point to another business's customer. Only an **owner** can change roles; nobody can promote themselves. The audit log can be added to but never edited or deleted.
Tamper-proof invoices (GoBD)	Once an invoice is "locked" it can no longer be edited or deleted, and the lock time is stamped automatically. The invoice number can never be changed.
Onboarding function	A new function `create_business_for_user()` creates the business *and* its owner in one go (both or neither). Apps cannot insert into those tables directly.
Invoices for brand-new customers	A document can now store a **copy of the recipient** (name and address) instead of needing a customer record. That also freezes what was printed even if the customer is edited later.
Sanity checks	Allowed statuses/roles are enforced; totals must add up (net + VAT = gross); "last changed" times update automatically.
Safe helper functions	The internal helpers were hardened and cannot be called by logged-out visitors.
File storage rules	Recordings and PDFs are private; each business can only touch files in its own folder.

Step 3 - Login and business setup done

Goal: a real front door for the app.

App starts
Is the cloud address configured? If not, it shows a short "how to configure" message.

▼

Not logged in?
→ Login / Register screen

▼

Logged in but no business yet?
→ "Set up your business" (two questions: business name, your name, plus trade)

▼

Everything set?
→ The main app with three tabs

This decision logic lives in one small piece called the **AuthGate** (the "bouncer"). It reacts live: log out and you are instantly back at the login screen.

- **Login screen:** e-mail and password, show/hide password, clear German error messages ("E-Mail oder Passwort ist falsch"), password at least 8 characters on sign-up, and a friendly note if e-mail confirmation is switched on.
- **Business details screen ("Betrieb" tab):** the info printed on every invoice - address, tax number, VAT ID, bank details, standard hourly rate and VAT rate. It checks input as you type: a real **IBAN checksum test**, VAT-ID format, sensible rates. It accepts German commas ("68,50").
- **Missing-information warning:** a yellow banner lists what an invoice still lacks (address, tax number/VAT ID, IBAN).
- **Roles:** only the owner and the office role can edit business data; others can view. (The database enforces this too - the app is just being polite.)
- **Price parser fixed:** typing "1.5" used to be understood as 15 cents. It now understands both German ("1.250,50") and plain ("65.50") ways of writing prices, and rejects nonsense.

Step 4 - Local database and sync done

Goal: real customers and jobs, working **without internet**.

ANALOGY: A NOTEBOOK AND A FILING CLERK

Thorsten carries a **notebook** (the phone's local database). He writes everything in it immediately, wherever he is. Every new note also goes into a **"to-send" tray** (the outbox). Whenever there is internet, a **filing clerk** (the sync engine) carries the tray's contents to the head office, and brings back any new papers from there. A note leaves the tray only after the office confirms it arrived.

WHAT YOU SEE

- **Aufträge (Jobs) tab:** list of jobs with customer and coloured status; a summary of drafts / in progress / completed; add, edit, delete.
- **Kunden (Customers) tab:** list with search (name, street, city, phone - several words allowed), add, edit, delete (asks first).
- **Sync tile in the Betrieb tab:** honest status - "Synchronisiert", "Offline", "Please sign in again", or "Some changes were rejected", with the last sync time and how many changes are waiting. Tap it to sync now.

HOW THE SYNC WORKS, IN A WORKED EXAMPLE

1. In a basement with no signal, Thorsten adds the customer "Schmidt". It appears in the list **immediately** and a note "upload customer Schmidt" lands in the outbox.
2. He then adds a job for Schmidt. A second note lands in the outbox.
3. Back in his van the phone gets signal. The sync engine wakes up. **Order matters:** customers are sent *before* jobs (a job pointing at a customer the cloud has not seen would be refused). Deletes go the other way round: jobs first, then customers.
4. Each note leaves the outbox only when the cloud confirms it. If the cloud *refuses* one (e.g. a rule is violated) that note stays, with the reason, and the others continue. If the connection drops, nothing is lost - it retries later.
5. Then the engine **pulls**: it asks the cloud "what changed since I last asked?" (a bookmark, called a *watermark*, remembers where it stopped), and also removes local rows that were deleted elsewhere.

WHEN DOES A SYNC HAPPEN?

When the app opens, a couple of seconds after any change, as soon as the network returns, every 5 minutes, and whenever you tap the sync tile. Only one sync runs at a time.

WHAT IF TWO PEOPLE EDIT THE SAME THING?

The rule is **"last write wins"**: whoever syncs last decides. A change still waiting in the outbox is *never* overwritten by incoming data. This is simple and fine for a small team, but two people editing the *same* customer at the same moment could overwrite each other.

TWO BUGS FOUND AND FIXED DURING THE BUILD

LOST EDIT

If you edited a record *while* the previous version was being uploaded, the app could have deleted your newer edit from the outbox when the older upload finished. Each outbox entry now carries a revision number, and it is only removed if that number is unchanged. A test proves it - and I confirmed the test fails when the protection is turned off.

ACCIDENTAL DELETION AT SCALE

The cloud returns at most 1,000 rows at a time. The "remove what was deleted elsewhere" step would have mistaken customer number 1,001+ for deleted ones. The list is now fetched page by page.

SHARED PHONE SAFETY

The phone remembers which business its cached data belongs to. If a *different* business logs in on the same phone, the old cache is wiped first so nobody sees someone else's customers.

TOOLING NOTE

Drift needs a "code generator" that writes helper code from the table definitions. It crashed at first because its internals were older than your Flutter version. I upgraded it and removed a second generator (`riverpod_generator`) that nothing used. The generated file is saved in the project so you do not need to run it again unless the tables change.

Step 5 - The real capture flow done

Goal: the "magic" step: speak (or type) a note, and get a draft - even with bad reception.

WHAT YOU SEE

1. Tap the amber **SPRECHEN** button. A sheet opens with a big microphone button. The first time, the phone asks for permission to use the microphone.
2. Tap to start. A timer counts up. Tap again to stop. (The limit is **3 minutes**: the cloud only reads the first 4,000 characters of the text, so longer notes would be cut off silently. 3 minutes of speech stays safely below that.)
3. The sheet closes at once with "Aufnahme gespeichert". You do **not** wait. The note appears at the top of the Aufträge tab under **NEUE NOTIZEN**.
4. A moment later it turns green: title, customer and "Entwurf prüfen" (check the draft). Tap it to open the review screen with the AI's draft and its warnings, for example "price estimated".

No microphone, or too noisy? The same sheet has **"Lieber tippen"** (type instead): the typed text goes through exactly the same pipeline, minus the speech recognition.

THE LIFE OF A NOTE

Status shown	Meaning	What happens next
Waiting (amber)	Saved on the phone, not yet processed. If there is no internet it says "Wartet auf Internet".	Sent automatically as soon as possible. Nothing for you to do.
Processing (blue)	Being uploaded and turned into a draft.	Usually seconds.
Ready (green)	The draft exists.	Tap to review. The small X removes the draft from the list (the job itself stays).
Failed (red)	Shows the reason in plain German.	**Wiederholen** (try again) or **Verwerfen** (discard).

> **ANALOGY: A PARCEL WITH A TRACKING NUMBER**
>
> The voice note is a parcel. The phone puts it in an **outgoing mailbox** (the queue). A **postman** (the processor) delivers it whenever the roads are open. Every parcel has a unique **tracking number**. If the postman delivers the parcel but the "delivered" receipt gets lost, he tries again next time - and head office says "we already have that one, here is the result again" instead of opening a second job and paying the AI twice.

BUILT TO SURVIVE INTERRUPTIONS

What goes wrong	What the app does
No internet when you stop recording	The note waits in the queue and is sent automatically when the network returns.
Signal drops *after* the audio was uploaded	The retry does **not** upload the audio again and uses the same tracking number, so no duplicate job appears.
The cloud has a temporary hiccup	Up to 3 automatic tries, then "failed" with a clear message and a retry button.
The AI could not make a usable draft	Fails straight away with the server's message (retrying the identical input rarely helps). You can try again or type it.
The app is closed or killed mid-processing	At the next start, unfinished notes are put back in the queue.
Login expired	Everything pauses and nothing is lost; the note waits until you sign in again.
You tap the mic by accident (under 1 second)	"Die Aufnahme war zu kurz" - nothing is queued.
You close the sheet while recording	The recording is cancelled, so the microphone is never left running.

The audio file is deleted from the phone only **after** the draft has been safely stored.

TECHNICAL DETAILS, IN PLAIN WORDS

- **Recording format:** speech is recorded mono, 16 kHz, AAC (a small compressed format): about half a megabyte per minute. Noise suppression and automatic volume are switched on for building sites.
- **Where the audio goes:** into private cloud storage, in a folder named after the business (`<business id>/<note id>.m4a`), matching the cloud function's folder rule.
- **Phone permissions:** Android needs `RECORD_AUDIO` declared; iPhone needs a sentence explaining why the microphone is used (otherwise the app would crash when asking). Both were added.
- **Database change on the phone:** a new table for queued notes. The local database moved from version 1 to 2 with a proper upgrade step, so existing installs keep their data.
- **Cloud change:** a third migration adds a "capture id" column with a uniqueness rule, and the Edge Function now recognises repeats (and handles two identical requests arriving at the same moment by keeping one result and removing the duplicate job).

BUGS THE TESTS CAUGHT WHILE BUILDING THIS

> **THE MICROPHONE COULD HAVE STAYED ON**
>
> If the sheet was dismissed while recording, the cleanup code used a Riverpod feature that is not allowed at that moment. It would have thrown an error and left the microphone recording. The test "closing the sheet while recording" found it; the code now takes what it needs at start-up.

> **DUPLICATE ITEM IDS**
>
> AI drafts have no item IDs, so the app invented them from the clock, in milliseconds. Several items created in the same millisecond got the *same* ID, which would have broken editing and deleting. Each item now gets a unique ID.

> **FILE ACCESS COULD NOT BE TESTED**
>
> Reading the audio file straight from disk meant the processing never finished inside a test (real disk access cannot run inside the test simulator's fake clock). File access now sits behind a small interface, like the microphone and the cloud, so each can be swapped for a pretend version in tests.

Step 6 - Real documents: finalize, number, PDF done

Goal: turn a checked draft into a legally usable quote or invoice: a gapless number, a frozen record, and a PDF you can send.

WHAT YOU SEE

1. **Review the draft.** Open a green note. Fix anything: the customer's name and address, each position (type: Arbeitslohn / Pauschale / Material, quantity with a comma like "1,5", price), add or delete positions. A yellow box lists the AI's warnings. The VAT line follows your business setting (19 %, 7 %, or 0 % for small businesses).
2. **Look at a preview** ("Vorschau als PDF"). It is clearly stamped **Entwurf** in red, shows "wird vergeben" instead of a number, and has no share button - so a draft can never be sent out by mistake.
3. **Tap "Rechnung fertigstellen".** A last-look box asks for the *Leistungszeitraum* (when the work was done), the payment term (7, 14 or 30 days; for quotes: valid 14, 30 or 60 days), whether to add a new customer to your list, and, if any price is still flagged "Preis prüfen", that you have checked it.
4. **The number is issued** (for example RE-2026-0001), the document is saved, the PDF is made and a copy is stored online. The PDF opens with **share** (WhatsApp, e-mail) and **print**.
5. **Tidy-up happens by itself:** the note disappears from NEUE NOTIZEN, the job moves to "Abgerechnet" (or "Angebot versendet"), and a new customer appears in your customer list - with the address split into street, postcode and city.

WHY YOU NEED INTERNET TO FINALIZE

> **ANALOGY: THE TICKET MACHINE AT HEAD OFFICE**
>
> Draft documents can be edited anywhere, even in a basement. But invoice numbers must be consecutive with **no gaps** (German tax auditors check this). Only one place can guarantee that: a single ticket machine at head office (the cloud). So the final step needs a connection. If it fails, the app says so, **keeps all your edits**, and you simply try again later.

HOW THE NUMBERS ARE TRULY GAPLESS

In Step 2 the number was handed out by one cloud call and the document was saved by a *second* call. If the connection died in between, a number would be used up with no document behind it: a gap. Now **one cloud function (`create_document`) does everything in a single transaction**: check the input, calculate the totals, take the next number, save the document. Either all of it happens or none of it does. Retrying the same request (same document ID) returns the same document and the same number, never a second one.

WHAT THE CLOUD CHECKS BEFORE IT ISSUES ANYTHING

Check	Why
Totals are calculated by the cloud, never taken from the phone	The phone cannot send wrong or tampered amounts. The VAT rate also comes from the business settings in the cloud.
An invoice needs the sender's full address and a tax number or USt-IdNr.	Required on every invoice by § 14 UStG. (The app checks the same thing first, so you get a clear message without waiting.)

Over 250 € gross, the customer's address is required	Small invoices up to 250 € may leave it out (§ 33 UStDV); larger ones may not.
Every position is valid: type, unit, text, quantity, price	No empty, negative or absurd positions. (Missing fields are caught too; see the bugs below.)
Customer and job must belong to your business	No mixing up with another business's records.
Dates are sensible	No invoice dated years ago or in the far future.
Documents cannot be inserted any other way	The old direct "insert" permission was removed; only this one function can create a document.

AFTER IT IS ISSUED

The content is **frozen** (GoBD): amounts, positions, recipient, number, dates, even the lock itself can never change, and it can never be deleted. Only two things may still change: the **status** (Erstellt → Versendet → Bezahlt, never backwards) and the PDF file path. For corrections you create a new document.

THE PDF, BEFORE AND AFTER

Before	Now
Letters like ä ö ü ß and € could drop out; spellings like "fuer" were a workaround	An embedded font (Lato, open licence): all German letters, € and § print correctly, with proper wording
Same invoice number, sample company and sample address on every PDF	The real number, your business data, the customer from the document
"19 % MwSt." fixed text	The real rate; with 0 % the line is replaced by "Gemäß § 19 UStG wird keine Umsatzsteuer berechnet."
Leistungszeitraum always the issue date	What you entered; due date or validity date included
Always one page	Long documents flow onto more pages, with "Seite 1 von 2" and the bank details in the footer
§ 35a note showed the net labour amount	Shows labour **including VAT** (the amount the deduction is based on), invoices only

I LOOKED AT THE RESULT

I did not just check that the PDF "exists". I rendered an invoice, a quote, a draft preview and a small-business invoice to images and looked at them: umlauts, ß, €, §, the numbers (for example 102,00 + 80,00 = 182,00 net, 34,58 VAT) and the layout were correct.

THE NEW BELEGE TAB

- **List** of all quotes and invoices, newest first, with colour-coded status and a summary: number of open invoices and the open amount.
- **Detail screen:** what the document says, status, and **"PDF ansehen und senden"**. It fetches the *original* PDF from storage; if that is not possible it recreates the PDF from the stored data.
- **Status buttons:** "Als versendet markieren" and (invoices only) "Als bezahlt markieren". Sharing the PDF also marks it as sent. Status changes need internet because the cloud keeps the master copy.
- Documents are kept up to date on the phone by the same sync as customers and jobs (read-only: they are only ever created by the cloud function).

MISTAKES FOUND AND FIXED WHILE BUILDING THIS

A HOLE IN MY OWN VALIDATION

In database language, "missing" is not the same as "wrong": a check like *type NOT IN (...)* silently passes when the type is missing altogether. A request with no positions at all could have produced an empty invoice. Every field is now checked for "missing" explicitly, and the tests try each missing field one by one and confirm that nothing is stored and no number is used up.

A MISTAKE FROM STEP 2

I had frozen locked documents so completely that nobody could mark an invoice "sent" or "paid". The rule now freezes the content but still allows the status workflow (migration 4 fixes this).

DUE DATES OFF BY A DAY

Computing "today + 30 days" as 30 × 24 hours is wrong when the clocks change (last Sunday of October): the due date lands one day early. Now it counts calendar days. A test crossing the clock change caught it.

A SPINNER BEHIND THE ERROR MESSAGE

When finalizing failed, the progress spinner kept turning behind the error box until you closed it. The busy state is now reset before any message is shown.

A NEWLY ISSUED DOCUMENT COULD VANISH FOR A MOMENT

Sync removes local documents that the server no longer lists. If you issued a document *while* a sync was running, that list was out of date and the new document was removed locally (it came back at the next sync). Sync now never removes a document that is newer than what it has already seen.

A QUIRK OF THE DATABASE LANGUAGE

A *CASE* … *THEN* inside an *IF* … *THEN* condition is cut off at the first THEN. Found by the tests; the value is now computed first.

7. How the app works

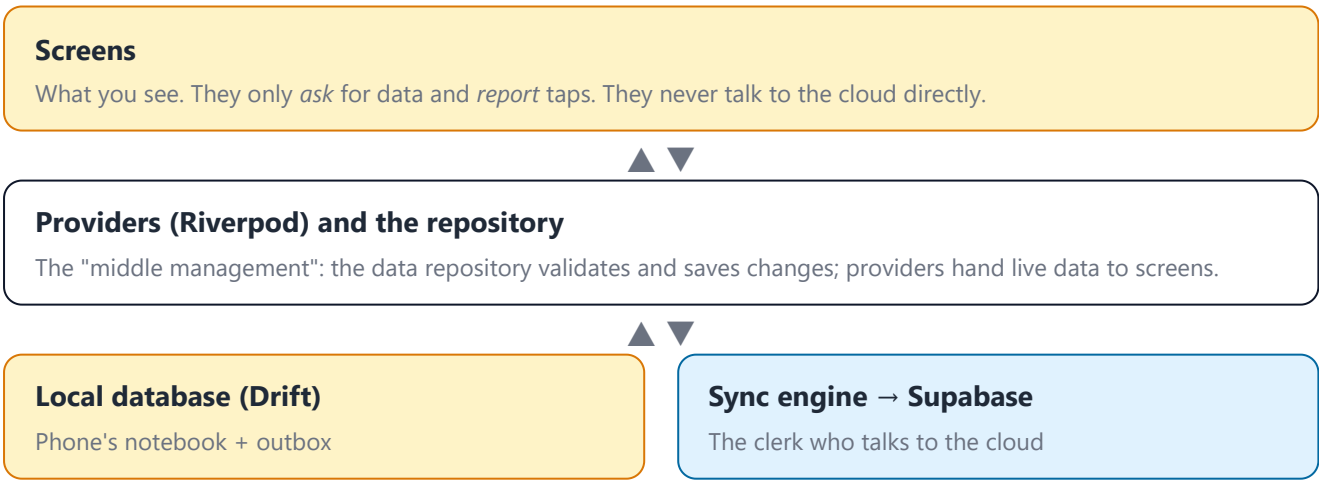
This chapter ties it together: what each part does when you use the app.

The three tabs

Tab	Shows	Data comes from
Aufträge	Your jobs, status chips, a small summary - and, at the top, **NEUE NOTIZEN**: voice and typed notes that are waiting, being processed, ready or failed.	Phone's local database (kept in sync with the cloud). Notes come from the phone's queue.
Kunden	Your customers, search.	Phone's local database.
Belege	All issued quotes and invoices with status, open invoices and amount; tap one for details, PDF and status buttons.	Phone's local copy, pulled from the cloud by sync (read-only).
Betrieb	Your business details, sync status, log-out.	Business details come from the cloud; sync status from the sync engine.

The big amber **SPRECHEN** (speak) button is always visible. It opens the real recording sheet (Chapter 6, Step 5). Tapping a finished note opens the review screen, where "fertigstellen" issues the numbered document (Chapter 6, Step 6).

Three layers inside the app



WHY THIS DESIGN?

Screens always read from the **local** notebook, so they are instant and never "wait for the internet". Only the sync engine deals with the network. This is also what made it testable: in tests we swap the real cloud for a pretend one.

What happens when you save a customer

1. You tap **Speichern**. The form checks the name is not empty.
2. The repository trims spaces, turns blank fields into "empty", creates a unique ID (a long random code - created on the phone so it works offline), and writes the customer **and** the outbox note in one all-or-nothing step.
3. The list updates instantly (screens watch the local database).
4. A sync is scheduled a moment later. If the network is there, the customer reaches the cloud; if not, it waits in the outbox.

What happens at login

1. Supabase checks e-mail and password and gives the app a login token (stored safely on the phone, so you stay logged in).
2. The app asks: "does this user have a profile?" If not → business setup. If yes → main app.
3. The app checks the local cache belongs to this business (wipes it otherwise), then starts syncing.

8. The money and tax rules inside the app

Labour vs. material (§ 35a)

Every line on a quote or invoice has a type:

Type	Examples	Counts for the customer's tax deduction?
labor	"2 hours: install socket"	Yes
flat_fee	"Call-out fee"	Yes (our decision - please verify)
material	"Gira double socket, 40 EUR"	No

The invoice shows a note for private customers: the labour amount **including VAT** (the deduction is based on the gross figure), the net amount, the material amount (stated as not eligible), and a reminder to **pay by bank transfer and keep the receipt** - cash payments are not deductible.

A worked example

Line	Type	Net
Installing socket, 2 h x 68,00 EUR	labor	136,00 EUR
Gira double socket	material	40,00 EUR
Net total		**176,00 EUR**
VAT 19 %		33,44 EUR
Gross total		**209,44 EUR**
Labour including VAT (for the tax note)		161,84 EUR

This exact example is used in our tests on **both** the phone side and the cloud side, so the two always agree.

Other rules

- **Money in cents.** 65,00 EUR is stored as 6500 so rounding errors cannot creep in.
- **Invoice numbers** are sequential per business, type and year and gap-free (Chapter 6, Step 6). Quotes use `ANG-`, invoices `RE-`. The number is issued in the same cloud step that saves the document, only when it is finalised, never for drafts.
- **The cloud calculates the final amounts** from the positions; the phone only shows a preview with the same rules.

- **Small businesses (§ 19 UStG):** with a VAT rate of 0 % the invoice carries the mandatory sentence instead of a VAT line.
- **Invoices over 250 €** must show the customer's address; small ones may leave it out.
- **Finished invoices are locked** (GoBD).
- **VAT rate** is a business setting (19 % default).

9. Security

A business app holds customers' names, addresses and money. Here is the protection, from the outside in.

Layer 1 - Login.
No wristband, no entry. Passwords are handled by Supabase, never stored by us.

▼

Layer 2 - The Edge Function doorman.
Works out the business from the profile; limits file sizes; only allows files from the business's own folder.

▼

Layer 3 - Database locks (RLS).
Even with a valid login, you only see and change rows of your own business. Cross-business links are refused.

▼

Layer 4 - Rules inside the database.
Roles cannot be self-promoted; documents can only be created through one controlled function; finished invoices are frozen; the audit log is append-only.

THE GOLDEN RULE ABOUT SECRETS

Only two kinds of keys exist: the **public** key (safe inside the app, it only allows what the locks permit) and the **secret** keys (Gemini, Deepgram, the master key). Secret keys live *only* in the cloud. The `.gitignore` also prevents secrets from being saved into Git by accident.

NOT YET DONE

Self-signup is currently open (fine for development, review before launch). The app's local phone database is not encrypted beyond what the phone itself provides.

10. The files

You do not need to read code, but it helps to know where things live.

```
D:\Project_Alpha_X
|-- ZUNFT_BUILD_PLAN.md          the original plan and product brief
|-- README.md                    how to start, how to run tests
|-- .env.example                 which secret settings exist (no real secrets)
|-- docs\                        this guide
|
|-- supabase\                    THE CLOUD
|   |-- config.toml              local Supabase settings
|   |-- seed.sql                 demo data for a local test database
|   |-- migrations\              database changes, in order
|   |     20261001...init_zunft_schema.sql     the original tables
|   |     20261002...harden_schema.sql         locks, numbering, storage rules
|   |     20261003...capture_idempotency.sql   duplicate-note protection
|   |     20261004...document_workflow.sql     create_document(), status workflow
|   |-- functions\
|   |     process-capture\index.ts             the "head chef"
|   |     _shared\draft.ts, draft_test.ts      AI-answer checker + totals (+ tests)
|   |-- tests\schema.test.mjs                  33 database tests
|
|-- zunft_app\                   THE PHONE APP
    |-- android\ ios\ windows\   per-platform wrappers
    |-- assets\fonts\           Lato font for the PDF (open licence)
    |-- lib\
    |   |-- main.dart            app start
    |   |-- app.dart             the three-tab main screen
    |   |-- core\
    |   |   |-- config\env.dart            cloud address settings
    |   |   |-- database\app_database.dart the phone's notebook + outbox
    |   |   |-- sync\                      the "filing clerk" (engine, remote, controller)
    |   |   |-- theme\ utils\ constants\   colours, price parsing, validators
    |   |-- features\
    |       |-- auth\            login, onboarding, business details
    |       |-- customers\       customer list + edit form
    |       |-- jobs\            job list + edit form
    |       |-- data\            repository + live data providers + sync tile
    |       |-- capture\         recording sheet, note queue, processor
    |       |-- review\          draft review screen
    |       |-- documents\      finalize service, PDF, Belege tab
    |-- test\                    162 app tests
```

11. Testing - how we know it works

A **test** is a little program that uses another program and checks the result. If we change something later and break it, tests shout immediately. **205 tests** currently pass.

Where	Count	What they check
App (Flutter)	162	Price formatting and parsing; tax split and totals; PDF is created; form checks (IBAN, VAT ID, e-mail, rates); login, registration and onboarding screens; business details editing; sync engine (push order, offline, rejected changes, deletes, paging, lost-edit protection); customers and jobs screens; the sync tile; the note queue (upload, retries, offline, duplicates, vanished files); the recording sheet (permission, too short, 3-minute limit, closing mid-recording, typed notes); the NEUE NOTIZEN list; the PDF (fonts, all variants, several pages); the finalize service (numbers, retries, offline, rejections, housekeeping, status, sync of documents); the review and finalize screens; the Belege tab.
Cloud function (Deno)	10	Totals match the app; flat fee counts as labour; the AI's mistakes are caught (bad types, negative numbers, invented customer IDs); a repeated request gets the stored result back.
Database (SQL)	33	Businesses cannot see each other; cross-links refused; role protection; audit log cannot be edited; invoice numbers sequential, per business, gap-free after a failed save; locked invoices frozen; storage folder rules; the same note id cannot be logged twice in one business; create_document (totals computed by the cloud, retries, every invalid or missing field rejected, nothing stored and no number used up on rejection, the 250 EUR rule, business completeness, foreign references, no direct inserts); frozen content with the status workflow; the insert guard.

WE LOOKED AT THE OUTPUT, TOO

For the PDF I rendered real samples and inspected them as images (Chapter 6, Step 6), because "a valid PDF file" says nothing about how it looks.

WE TESTED THE TESTS

Tests that always pass prove nothing. For the most delicate protection (the lost-edit guard) I switched it off on purpose and confirmed the test then fails, then switched it back on.

What we could not check - important

NOT VERIFIED AGAINST THE REAL THING

- **No real Supabase yet.** This computer has no Docker and no Supabase tool, so the database tests used a look-alike engine, and the sync tests used a pretend cloud. The *logic* is thoroughly tested; the *real connection* (exact web-request details, timestamps) is the first thing to check on first run.
- **Never run on a phone or emulator.** Tests render screens in a simulator with a *pretend microphone*. Whether the real microphone, the permission pop-ups on Android and iPhone, and the audio file format work has not been checked on a device.
- **Gemini and Deepgram never called.** We have no keys yet, so nobody has seen how well the AI really turns German speech into drafts. The prompt wording and the model name (`gemini-2.5-flash-lite`) must be confirmed against Google's current list.
- **Tax rule from memory** (flat fee = labour). Verify before real use.

12. What is still open

Item	Plain explanation	When
Finalizing needs internet	The number comes from the cloud, so a document can only be issued online. Drafts can be prepared offline.	By design
No cancellation flow	You cannot cancel an issued invoice in the app. Legally a correction needs a cancellation invoice (Stornorechnung); today you create a new document instead.	Later
Original PDF vs. recreated PDF	The original PDF is stored online. If it cannot be downloaded, the app recreates it from the stored data using your *current* business details, so it could differ slightly if you changed them since.	Later
E-invoices (XRechnung / ZUGFeRD)	As I understand it, businesses in Germany must be able to *receive* e-invoices since 2025 and must *issue* them to other businesses from 2027 or 2028, depending on turnover. Invoices to private customers are not affected. Not built yet (planned "Phase 4"). Please verify the dates.	Phase 4
Invoice layout review	The invoice follows § 14 UStG as I know it, but I am not a tax adviser. Have someone check one real sample before you send invoices to customers.	Before launch
Reminders (Mahnungen)	The app shows open invoices but sends no reminders.	Later
Data protection (DSGVO)	Voice recordings and customer names are sent to outside companies (Deepgram, Google). Before real customers use it you need data-processing agreements with them, a privacy policy, and ideally EU data regions.	Before launch
Recording lost if the app dies	A note is only queued when you tap stop. If the phone crashes or the battery dies *during* a recording, that recording is lost.	Later
Deploying the function	The Edge Function and the four migrations (four now) must be deployed to a real Supabase project, with the secret keys set there.	First real test
Adding staff	Only the owner exists. An invite feature for employees is not built.	Later
Edit conflicts	"Last write wins" can overwrite simultaneous edits to the same record.	Later, if needed
No automatic fix for rejected changes	The tile tells you something was rejected but offers no "resolve" screen.	Later
First "save point"	The first commit is made (local only). Pushing to GitHub comes when you ask.	Done locally

Windows Developer Mode	Needed on this PC to run the app locally. Turn it on once via `start ms-settings:developers`.	You

13. What comes next

Step 5 - The real capture flow done

Built (see Chapter 6). The first real test needs the keys and a deployed cloud project, listed below.

Step 6 - Documents done

Built (see Chapter 6).

Step 7 - Quality and release

1. Test the voice-to-draft quality on about 25 real German voice notes (this is where we decide on Deepgram vs. Gemini audio and which Gemini tier).
2. Automatic checks on every change (CI), more tests.
3. Test on real devices; prepare the store releases.

What you will need to provide

Item	What it is	Needed for
Supabase project	A free account at supabase.com, or Docker on this PC for a local copy.	Running the cloud for real
Gemini API key	From Google AI Studio.	The first real test (now possible)
Deepgram API key	From deepgram.com (new accounts get free credit).	The first real test (now possible)
An Android phone (or emulator)	To tap through the app.	All device testing
A Mac with Xcode	Apple only allows iPhone apps to be built on a Mac.	The iPhone version
Developer accounts	Google Play (one-time fee) and Apple Developer (yearly fee).	Publishing
10-25 real German voice notes	Recordings of the kind of thing a craftsman would actually say - ideally from a real electrician.	Judging AI quality
A tax check	When you can: have someone confirm the § 35a handling.	Before real invoices

14. Your questions, answered

You asked these as "unguided observations". Most of them are good ones. Each answer says what is true **today**, and whether it should become a job.

Is Zunft for one tradesman or for small companies?

Both, but today only the "boss" side exists. The database was designed for a business with several people (every person has a profile that points to a business, with a role). But the app currently only has screens for the owner. There is no screen to add employees, and no time tracking.

> **MY RECOMMENDATION**
>
> Do not build a time-tracking system or a desktop program yet. Time tracking is a whole separate product area (existing tools such as Clockodo or Toggl already do it well). First prove the core idea: voice note becomes invoice. Later, the cheap way to give the boss an overview is a web page version of this same app (Flutter can do that) plus a "download as spreadsheet" button that existing time tools can read. One app, not two.

Are we allowed to send customer data to outside companies?

I am not a lawyer, so treat this as a briefing and not as legal advice. As I understand the GDPR (DSGVO):

- **It is allowed** to use outside companies to process data, if you have a signed **data-processing agreement** (*Auftragsverarbeitungsvertrag*, AVV) with each one, your customers can read a **privacy policy**, and any transfer outside the EU has a legal safeguard.
- **Today we use two such companies**: Deepgram (voice to text) and Google (Gemini). Both are US companies. Both offer the paperwork, but you must actually accept it in their dashboards.
- **Free versus paid matters.** As far as I know, Google's *free* Gemini tier may use your data to improve its products. For real customers, use the **paid** tier (billing turned on), where Google says it does not train on your data. Please confirm this in Google's current terms.

Idea	Does it help?
Use **Gemini for the audio too** (it can listen)	Yes: one outside company fewer, and less paperwork. We will compare quality and cost in the voice-note test.
Use **Mistral** (French company)	It can help with "European" paperwork. As I know it, Mistral also offers an audio model (Voxtral). We would test it the same way. It is not automatically better: what matters is the contract, the data region, and quality in German trade language.

Choose an **EU data region** (for example Google Vertex AI in Europe)	Yes, this is probably the strongest single step. It needs a slightly different setup, which I can build once you decide.
Hide personal data from the AI	Partly. What we send now: the spoken text (which contains whatever the craftsman says, often a name) and a list of customer *names, first names and cities*. We never send street addresses, phone numbers, bank details or e-mail. We can reduce it further: swap known customer names for codes ("Kunde 7") before the text goes to the AI, and match names on our own server. But the voice itself contains names, so the speech-to-text company always hears them. We can lower the amount of personal data, not make it zero.

WHAT TO DO

1) Use the paid Gemini tier. 2) Accept the data-processing agreement of each provider. 3) After the voice test, decide on one provider and an EU region. 4) Write a privacy policy (a lawyer or a template service). This is a launch requirement, not a blocker for testing with your own fake data.

Newer Gemini models (3.6, 3.8 Flash)?

I cannot confirm those model names from here: my knowledge may be older than yours. That is exactly why the model name is a **setting** (`LLM_MODEL`) and not written into the code. You change one secret and deploy again; no code changes. In Google AI Studio, open the model list, copy the exact ID of the newest Flash model, and put it in. We will pick the final model with the voice-note test, using quality per euro as the measure.

How does a new employee get an account?

Not built yet, and your instinct is right that it must be solved. One rule I would keep: the boss never shares their own login. Every person gets their own account so that you can see who did what, and so that you can switch off one person (a leaving employee) without changing everyone's password.

1. Boss opens "Team" and types the employee's e-mail

↓

2. The cloud sends an invitation e-mail

Done by a server function, because creating accounts needs the secret key, which never goes in the app.

↓

3. Employee taps the link, chooses a password, and lands inside the boss's business

With the role "employee": can record and see jobs, cannot change business settings.

Decision for you later: may employees see *all* invoices and customers, or only their own jobs? Today everyone in a business would see everything.

Fingerprint and face login

Yes, easy and worth doing. It does not replace the account. The phone keeps you logged in, and fingerprint/face is simply a lock that opens the app (Android and iPhone both support it). It needs a real phone to test properly, so I want to add it right after the first real test and not before.

Worker goes from job to job with the internet off

This is what the queue was built for. Every recording gets its own unique number and is saved on the phone. When the internet returns, the app uploads them one by one, in order. A failed upload retries three times, and a real rejection is shown clearly and nothing silently disappears. Customers and jobs typed offline sync the same way.

Honest limits: (1) **issuing** the final numbered invoice needs internet; (2) a recording is only queued when you tap stop, so a phone crash *during* a recording loses it; (3) I have tested all this with simulated networks, not with a real phone in a real basement. That is what the first real test is for.

"Each worker has their own customers, so two people can't collide"

Small correction: customers belong to the business, not to a worker. Two employees see the same customer list. Two good things: recordings can never collide (each phone has its own queue and every note becomes its own draft). Two weak spots: two people may create "Herr Müller" twice (a duplicate), or edit the same customer at the same time (the last save wins). A "merge duplicates" function is a sensible later feature.

Is the voice recording kept, for checking the AI?

Partly. The text the AI heard (the transcript) is saved with every job, and the audio file stays in the cloud storage. Only the copy on the phone is deleted once the draft exists. **Missing:** there is no "play recording" button in the app yet. This is a good, small feature: on the review screen, show the transcript and a play button, so the craftsman can check what was understood. I will put it on the list. (Under the GDPR you should also decide how long audio is kept; I suggest deleting it after a set time, for example 90 days.)

Do 10 notes for Herr Müller all land on one invoice?

No, and this is the opposite of what you feared. Today every voice note becomes its own separate draft. Nothing is added onto an existing invoice, so a tired worker cannot accidentally put Frau Neumann's work on Herr Müller's invoice.

But your question reveals a **real gap**: if Thorsten sends 10 notes for the same customer in one day, he gets 10 drafts, when he probably wants one invoice. And the AI guesses the customer from the spoken name. Both are worth fixing like this:

- Start a recording **from inside a job**, with a big, clear banner "Aufnahme für: Müller, Badsanierung". (The cloud function can already attach a note to a job; the app does not use that yet.)

- On the review screen, show the customer the AI picked at the top in big letters, with a one-tap "wrong customer? change".
- A button "Add to existing draft" to merge several notes into one invoice.

My recommended order

#	What	Why this order
1	**First real test** (Chapter 15)	Everything so far is only tested with simulations. Real problems will show up here.
2	Voice-note quality test, then choose provider and EU region	Settles Deepgram vs. Gemini vs. Mistral and the privacy design in one go.
3	Job-linked recording, customer check, play button, merge notes	Fixes the real usability gaps you spotted.
4	Team invites and roles	Needed as soon as a second person uses it.
5	Fingerprint / face lock	Small, best tested on a real phone.
6	Boss overview, time entries, web version	Only after the basics are loved by a real customer.

15. Your next steps: a simple checklist

GOOD NEWS

You do not need to understand any of this. Do the steps in order, tick them off, and tell me when you are stuck or done. Anything that goes wrong: copy the red text and send it to me. Developer Mode is already on, so that part is done.

Part A - Create three free accounts (about 20 minutes)

1. **Supabase** (the cloud database). Go to `supabase.com`, sign up, click **New project**. Name: `zunft`. Choose a **region in Europe (Frankfurt)**. Write down the **database password** you choose. Wait about 2 minutes until it is ready.
2. **Collect 3 values from Supabase.** (a) *Project URL* and the *publishable (anon) key*: Project Settings → API. (b) *Project reference*: the short text in the URL, `https://XXXX.supabase.co`, that is the XXXX. Keep them in a text file. **Never** send me the "service_role" or "secret" key.
3. **Gemini key.** Go to `aistudio.google.com` → **Get API key**. For the test, the free tier is fine with fake data. Later, switch on billing before using real customer data.
4. **Deepgram key.** Sign up at `deepgram.com` (free starting credit) and create an API key.

Part B - Put the cloud part online (about 15 minutes)

Open a terminal in the folder `D:\Project_Alpha_X` and run these one at a time. (In Claude Code you can type `!` before a command to run it. The first command may ask to install a small helper: say yes.)

5. `npx supabase login` — a browser opens; allow it.
6. `npx supabase link --project-ref XXXX` — use your project reference. It asks for the database password from step 1.
7. `npx supabase db push` — builds the tables (four migrations). Answer **Y**.
8. `npx supabase secrets set GEMINI_API_KEY=your-key DEEPGRAM_API_KEY=your-key LLM_MODEL=gemini-2.5-flash-lite` — the secrets stay in the cloud; nobody sees them.
9. `npx supabase functions deploy process-capture` — puts the AI function online.

IF SUPABASE ASKS FOR DOCKER

It is only needed for a local copy. For the online project you do not need it. Tell me what it said and I will adjust.

Part C - Run the app on a phone (about 20 minutes)

10. **Android phone:** Settings → About phone → tap *Build number* 7 times → back → Developer options → turn on **USB debugging**. Plug it in with a cable and accept the prompt on the phone. (No phone? Tell me; we use an emulator from Android Studio.)
11. In the terminal: `cd zunft_app`, then `flutter devices`. Your phone should be listed.
12. `flutter run --dart-define=SUPABASE_URL=https://XXXX.supabase.co --dart-define=SUPABASE_ANON_KEY=your-publishable-key` — the first build takes several minutes. That is normal.

Part D - The real test (about 30 minutes)

13. In the app: register, then fill in the business details (invent them or use real ones; use a **fake** tax number for now).
14. Tap record and say, in German: "*Kunde Müller, Hauptstraße 5 in Hamburg. Steckdose im Keller gesetzt, zwei Stunden Arbeit, Material etwa vierzig Euro.*"
15. Wait for the draft. Check the customer, the labour/material split and the sums. Press **Vorschau als PDF**, then **Fertigstellen** and look at the PDF.
16. **Airplane-mode test:** turn on airplane mode, record two more notes for different customers, then switch the internet back on. Both drafts should appear on their own.
17. Send me: what worked, what did not, any red error text, and a photo of the PDF if it looks wrong. Then record **10 to 25 more** voice notes the way a real craftsman talks (ideally have a real one record some) and keep the audio files; they are the test set for choosing the AI.

STUCK? THAT IS NORMAL

The first run of any app against a real cloud almost always has one or two surprises (a wrong key, a missing setting). Send me the exact message and we fix it together. You cannot break anything permanent: the project is empty.

PARALLEL, NO RUSH

Look into: a data-processing agreement with each provider (Chapter 14), a privacy policy, someone to check the invoice layout and the § 35a handling, and an Apple Developer account plus a Mac if you want the iPhone version.

SUMMARY IN ONE PARAGRAPH

Zunft is a phone app that turns a spoken job note into a legally usable German invoice. So far we have made the cloud side safe, built login and business setup, a local database with offline sync, the recording queue that turns a note into a draft even with bad reception, and now the last stretch: checking the draft, issuing a numbered and locked document, and a proper PDF - all verified by 205 automatic tests. Still ahead: a first real test with real keys and phones, a review of the invoice layout, then release (Step 7).